

Feladatsor: JavaScript, AJAX és Aszinkron működés

1. **Mi a különbség a POST és a GET metódus között a projektben használt API végpontoknál?**
 - **Megoldás:** A GET adatokat kér le a szerverről (pl. a személyek listáját), míg a POST új adatokat küld a szervernek feldolgozásra vagy mentésre (pl. új személy felvétele vagy köszöntés). A GET kérések paraméterei általában az URL-ben látszódnak, így korlátozott mennyiségű adatot tudnak átvinni. Ezzel szemben a POST az adatokat a kérés törzsében (body) küldi el, ami biztonságosabbá és alkalmassá teszi nagy adatcsomagok, például teljes űrlapok továbbítására.
2. **A server.js-ben miért van szükség az app.use(express.json()) sorra?**
 - **Megoldás:** Ez a middleware felelős azért, hogy a kliens által küldött JSON formátumú adatokat a szerver értelmezni tudja, és beillessze a req.body objektumba. Enélkül a szerver csak egy nyers szövegfolyamot látna, és nem tudná objektumként kezelni a beküldött nevet vagy email címet. A folyamat során az Express automatikusan elvégzi a JSON deszerializációt, így a fejlesztőnek már csak a kész változókcal kell dolgoznia.
3. **Magyarázd el, mi történik a fetch() hívás során, ha a szerver nem érhető el! Melyik blokk fogja lekezelni ezt a hibát a script.js-ben?**
 - **Megoldás:** A fetch() ígéretet (Promise) ad vissza. Ha a hálózati kapcsolat megszakad, a Promise "rejected" állapotba kerül, amit a try...catch blokk catch ága fog elkapni és naplózni. Ez a mechanizmus megakadályozza, hogy a program összeomoljon egy hálózati hiba miatt. Fontos tudni, hogy a fetch csak valódi hálózati hiba esetén dob kivételt, egy 404-es vagy 500-as HTTP hibakódnál még a try ágban maradunk.
4. **A script.js-ben a tablazatFrissites függvény async kulcsszóval van megjelölve. Mit jelent ez, és miért van rá szükség?**
 - **Megoldás:** Az async jelzi, hogy a függvényen belül aszinkron műveletek (pl. hálózati kérés) történnek, és lehetővé teszi az await használatát, így a kód olvashatóbb, lineáris marad. Az ilyen függvények mindig Promise-t adnak vissza, függetlenül attól, hogy van-e bennük explicit visszatérési érték. Segítségével elkerülhetjük a bonyolult visszahívási (callback) láncokat, és úgy írhatunk aszinkron kódot, mintha az szinkron módon, soronként futna le.
5. **Mi a szerepe az await kulcsszónak a const res = await fetch(...) sorban? Mi történne, ha elhagynánk?**
 - **Megoldás:** Az await várakozik a Promise feloldására (a válasz megérkezésére). Nélküle a változóba nem a válasz, hanem maga a függőben lévő Promise objektum kerülne, és a kód továbbfutna az adatok megérkezése előtt. Ez azt eredményezné, hogy a következő sorban, ahol az adatokat próbáljuk feldolgozni, hibát kapnánk, mert a válasz objektum még nem létezik. Az await tehát szünetelteti a függvény futását, amíg a háttérben zajló művelet be nem fejeződik.
6. **A projektben a módosításnál PUT, a törlésnél pedig DELETE metódust használunk. Milyen technikai előnye van ennek a sima POST használatával szemben?**

- **Megoldás:** Ez a RESTful API alapelve: a metódus kifejezi a szándékot. Segít a kód rendszerezésében, és bizonyos esetekben (pl. cache-elés vagy proxy) hatékonyabb és szabványosabb működést tesz lehetővé. A szemantikus metódusok használata megkönnyíti a szerveroldali útvonalválasztást (routing), hiszen ugyanaz az URL más funkciót tölthet be `GET` és `DELETE` esetén. Emellett a fejlesztőcsapat többi tagja számára is azonnal egyértelművé válik a végpont célja a kód elolvasásakor.
- 7. **Hogyan küldi el a kliens az adatokat a szervernek, hogy a `server.js` a `req.body` objektumon keresztül el tudja érni azokat?**
 - **Megoldás:** A kliens a `fetch` során a `headers` részben megadja a `'Content-Type': 'application/json'` értéket, a `body` részben pedig elküldi a stringgé alakított adatokat. Ez az információs fejrész árulja el a szervernek, hogy hogyan kell értelmeznie a beérkező adatfolyamot. Ha elhagynánk a fejléceket, a szerver nem tudná, hogy JSON-ról van szó, és a `req.body` üres maradna. Ez a szabványos módja az adatok strukturált átvitelének a modern webes alkalmazásokban.
- 8. **A `mysql2/promise` használata során miért kényelmesebb a `const [rows] = await pool.query(...)` forma, mint a hagyományos `callback`-alapú megoldás?**
 - **Megoldás:** Elkerülhető a "callback hell" (egymásba ágyazott függvények), a kód átláthatóbb lesz, és a hibakezelés egységesen megoldható a `try...catch` szerkezettel. A tömb-destrukurálás (`[rows]`) lehetővé teszi, hogy közvetlenül az eredményhalmazhoz férjünk hozzá, kihagyva a metaadatokat. Ez a megközelítés sokkal jobban illeszkedik a modern JavaScript "async/await" mintájához, mint a régi stílusú, mélyen egymásba ágyazott kódszerkezetek.
- 9. **Mi a célja a `JSON.stringify(adatok)` függvénynek az AJAX kérés összeállításakor?**
 - **Megoldás:** A JavaScript objektumot JSON formátumú szöveggé (string) alakítja, mivel a HTTP protokollon keresztül csak szöveges adatokat lehet küldeni. A hálózati réteg nem képes közvetlenül értelmezni a memóriában lévő bináris JavaScript objektumokat. A `stringify` művelet biztosítja, hogy az adatok egy univerzális, platformfüggetlen formátumban utazzanak a vezetéken. Amint megérkezik a szerverre, az ottani middleware végzi el az ellentétes műveletet, vagyis a szöveg visszalakítását objektummá.
- 10. **Hogyan akadályozza meg a `e.preventDefault()` sor a `szerelyForm` beküldésekor az oldal újratöltődését?**
 - **Megoldás:** Megállítja a böngésző alapértelmezett eseményét, ami a form beküldésekor az oldal frissítése vagy egy új URL-re való navigálás lenne, így az AJAX kérés a háttérben futhat le. Ezzel a módszerrel megőrizhetjük az alkalmazás aktuális állapotát, például a beírt szövegeket vagy a görgetési pozíciót. Ez a technika elengedhetetlen a "Single Page Application" (SPA) jellegű felületek létrehozásához, ahol a felhasználói élmény folyamatos marad.

Promise láncolat arról, hogyan vándorol az adat a hálózati kéréstől a képernyőig.

1. A Fetch folyamat lépései

Amikor kiadod a `fetch()` parancsot, egy többlépcsős "ígéret" (Promise) láncolat indul el.

```
fetch('/api/koszontes', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ nev: "Ajax" })  
})  
  
.then(response => {  
  // 1. lépés: Megérkezett a válasz fejléce (státuszkód, típus)  
  console.log("Státusz:", response.status);  
  return response.json(); // Itt indítjuk el a törzs feldolgozását  
})  
  
.then(adat => {  
  // 2. lépés: Itt már a kész, értelmezett JS objektum van a kezünkben  
  console.log("A kapott adat:", adat.uzenet);  
})  
  
.catch(error => {  
  // Hibaág: Ha a hálózat megszakad  
  console.error("Hálózati hiba:", error);  
});
```

2. Hol van az adat? (A változók magyarázata)

A `fetch()` hívás

A `fetch` azonnal visszatér egy **Promise** objektummal. Ez egy ígéret, hogy "majd lesz itt valami".

ebben a pillanatban az adat még nincs sehol, a kérés épp úton van a szerver felé.

Az első `.then(response => ...)`

- **Mikor fut le?** Amint a szerver válaszol (megérkezik a HTTP fejlécek).
- **Mi a `response`?** Ez egy `Response` objektum. Ez még **nem a tényleges adat** (pl. az üzenet), hanem a válasz "csomagolása". Tartalmazza, hogy sikeres volt-e a kérés (ok: `true`) vagy mi a hibakód (status: 200).
- **Mi történik itt?** Mivel a válasz törzse (a JSON) még olvasható folyamatként érkezik, le kell futtatnunk a `response.json()` metódust. Ez ismét egy Promise-t ad vissza.

A második `.then(adat => ...)`

- **Mikor fut le?** Amikor a böngésző befejezte a teljes szöveges válasz JSON-né alakítását (deszerializáció).
- **Mi az `adat`?** Ez már egy tiszta **JavaScript objektum**. Ez az a változó, amiben már benne vannak a szerveroldali `res.json()`-ben küldött kulcsok, például az `adat.uzenet` vagy `adat.id`.

3. A folyamat logikai íve (10 pontban)

1. **Indítás:** Meghívod a `fetch`-et.
2. **Promise 1:** A `fetch` ad egy "függőben lévő" (Pending) ígéretet.
3. **Hálózat:** Az adat elmegy a szerverre, az lefuttatja a SQL-t, majd válaszol.
4. **Megérkezés:** A böngésző megkapja a válasz elejét.
5. **Feloldás 1:** Az első Promise "teljesített" (Fulfilled) lesz, beugrik a `response` változóba.
6. **Átalakítás:** Meghívod a `response.json()`-t, ami elkezd feldolgozni a nyers adatfolyamot.
7. **Promise 2:** A `response.json()` ad egy újabb ígéretet a feldolgozásra.
8. **Kész adat:** Amint a feldolgozás kész, a második Promise is feloldódik.
9. **Feloldás 2:** Az eredmény belekerül az `adat` (vagy bármilyen általad elnevezett) változóba.
10. **Használat:** Ezen a ponton tudod kiírni a HTML-be:
`document.getElementById('valasz').innerText = adat.uzenet.`

Mi a különbség az `async/await`-hez képest?

A projektben használt `const res = await fetch(...)` pontosan ugyanazt csinálja, csak "megállítja" a kód futását az adott sorban, amíg az ígéret fel nem oldódik.

- A `.then(response => ...)` helyett ott a `const res = await fetch(...)` sor van.
- A `.then(adat => ...)` helyett pedig a `const adat = await res.json()` sor.
- A `res` változóban ott is a csomagolás (`Response`) van, az `adat` változóban pedig a tényleges tartalom (JS objektum).